

# ARM Cortex-M

White Paper ARM: Cortex-M for Beginners

Web DaveSpace

Introduction to ARM

(programación en lenguaje ensamblador)

Efficient C for ARM

(“improve your C so that it runs faster, takes up less space or possibly both”)



Empresa con base en Cambridge, UK fundada en 1990.

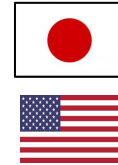
Arquitectura ARM de 32 bits (RISC) **licenciable**. No fabrican ni venden ICs. Entregan (deliverables) el core en RTL (register transfer level) sintetizable (por ejemplo Verilog). Modular, reconfigurable, herramientas.

Fue crucial para el desarrollo de sistemas portátiles de alta performance (smartphones). Android/iOS, **Cortex-A** (application) de 32 bits (2005) y 64 bits (2012).



**Cortex-M** (2005, embedded) de 32 bits..

En 2016 fue adquirida por el holding japonés SoftBank (32G\$)  
Desde 2020 NVidia (US) está intentando comprarla (40G\$)



China se opone (Huawei)

Googlear noticias 2021/22/23/24... no prosperó por ahora



### Empresas licenciatarias de ARM:

Apple, Intel, DEC, **NVIDIA**, Qualcomm, Samsung, NXP, IBM, Texas Instruments, Nintendo, Philips, **Huawei**, Sharp, Samsung, Atmel y STMicroelectronics.

## Silicon Partners



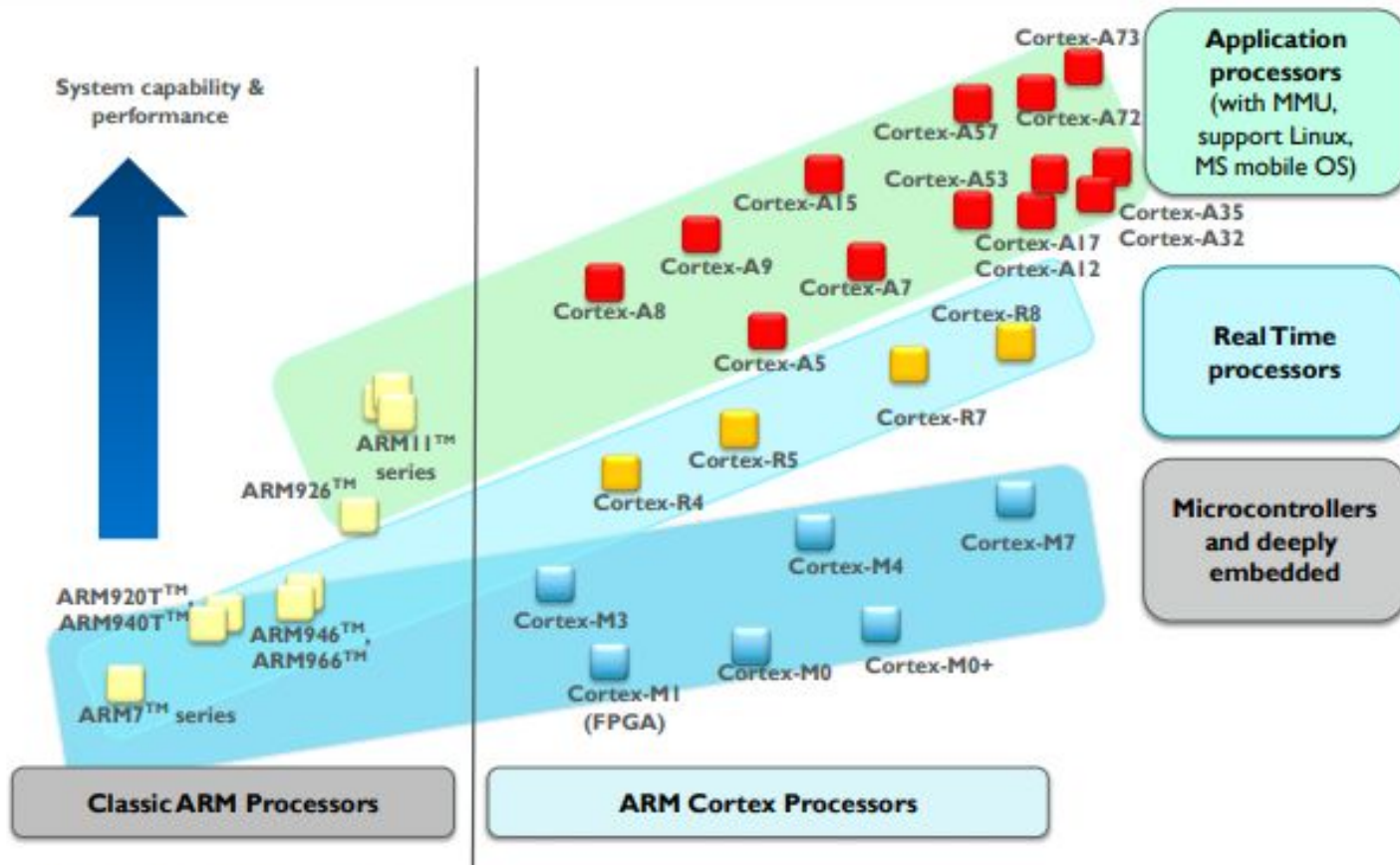
## Design Support Partners



## Software, Training and Consortia Partners



# Los procesadores de ARM



Conceptos de  
Sistemas Operativos (4º  
semestre)

Sistemas de Tiempo  
Real  
(8º semestre)

# ARM Cortex-A

Multicore

MMU (memory management unit)

Out-of-order execution (superescalar)

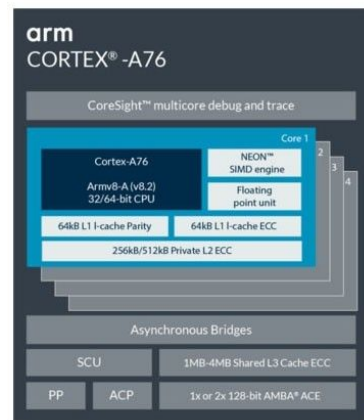
Deep pipeline

FPU (unidad de punto flotante)

Caches separados de instrucciones y de datos

SIMD (Neon)

Instruction set ARMv9 (64 bits)



## iPhone 16 (sept 2024)

Apple A18/A18pro

3 nm process by TSMC (N3)

6 cores (2 performance @4.04 GHz + 4 efficiency @2.42 GHz)

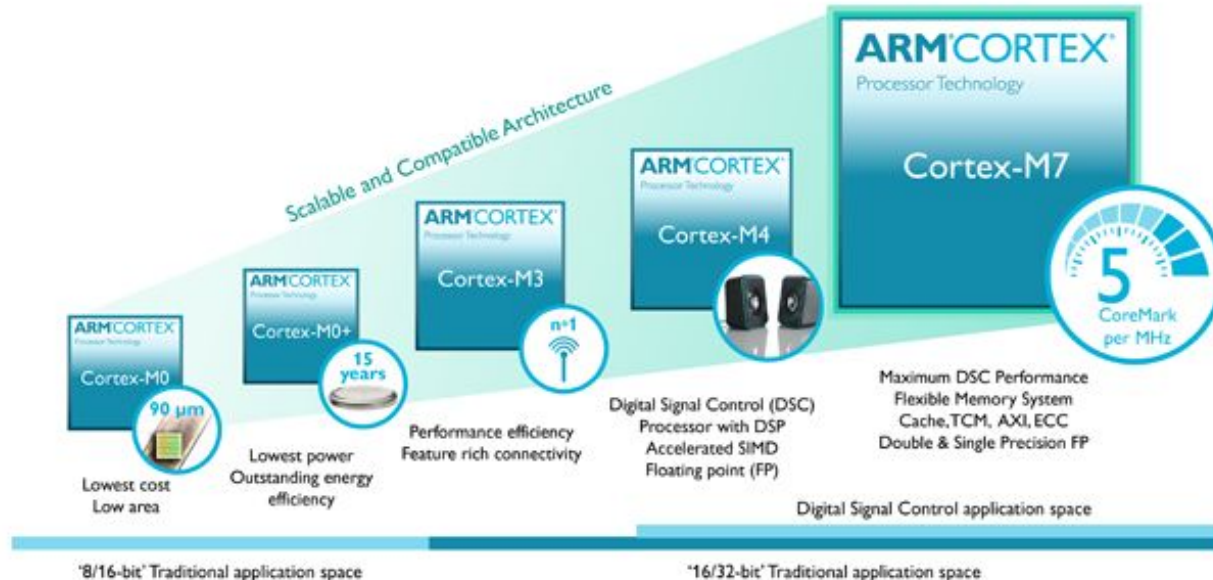
15/20 billion transistors



# ARM Cortex-M

## Procesadores ARM para aplicaciones embebidas

Son una familia de MCU (Micro-Controller Unit). **No tienen MMU (Memory Management Unit)**, necesaria para correr un sistema operativo de propósitos generales (por ejemplo Android en Cortex A). Tienen **MPU (Memory Protection Unit)** útil para RTOS.



### Wikipedia:

"Cortex-M have become a popular replacements for 8-bit chips in applications that benefit from 32-bit math operations."

### ARM Cortex-M Instruction Sets<sup>[6][7]</sup>

| ARM Cortex-M              | Thumb  | Thumb-2 | Hardware multiply | Hardware divide | Saturated math | DSP extensions | Floating-Point Unit (FPU) | ARM architecture |
|---------------------------|--------|---------|-------------------|-----------------|----------------|----------------|---------------------------|------------------|
| Cortex-M0 <sup>[1]</sup>  | Most   | Subset  | 1 or 32 cycle     | No              | No             | No             | No                        | ARMv6-M          |
| Cortex-M0+ <sup>[2]</sup> | Most   | Subset  | 1 or 32 cycle     | No              | No             | No             | No                        | ARMv6-M          |
| Cortex-M1 <sup>[3]</sup>  | Most   | Subset  | 3 or 33 cycle     | No              | No             | No             | No                        | ARMv6-M          |
| Cortex-M3 <sup>[4]</sup>  | Entire | Entire  | 1 cycle           | Yes             | Yes            | No             | No                        | ARMv7-M          |
| Cortex-M4 <sup>[5]</sup>  | Entire | Entire  | 1 cycle           | Yes             | Yes            | Yes            | Optional, SP              | ARMv7E-M         |
| Cortex-M7                 | Entire | Entire  | 1 cycle           | Yes             | Yes            | Yes            | Yes, SP & DP              | ARMv7E-M         |

**M4 DSP:** Harvard, Single cycle MAC, Barrel shifter, SIMD, aritmética con saturación

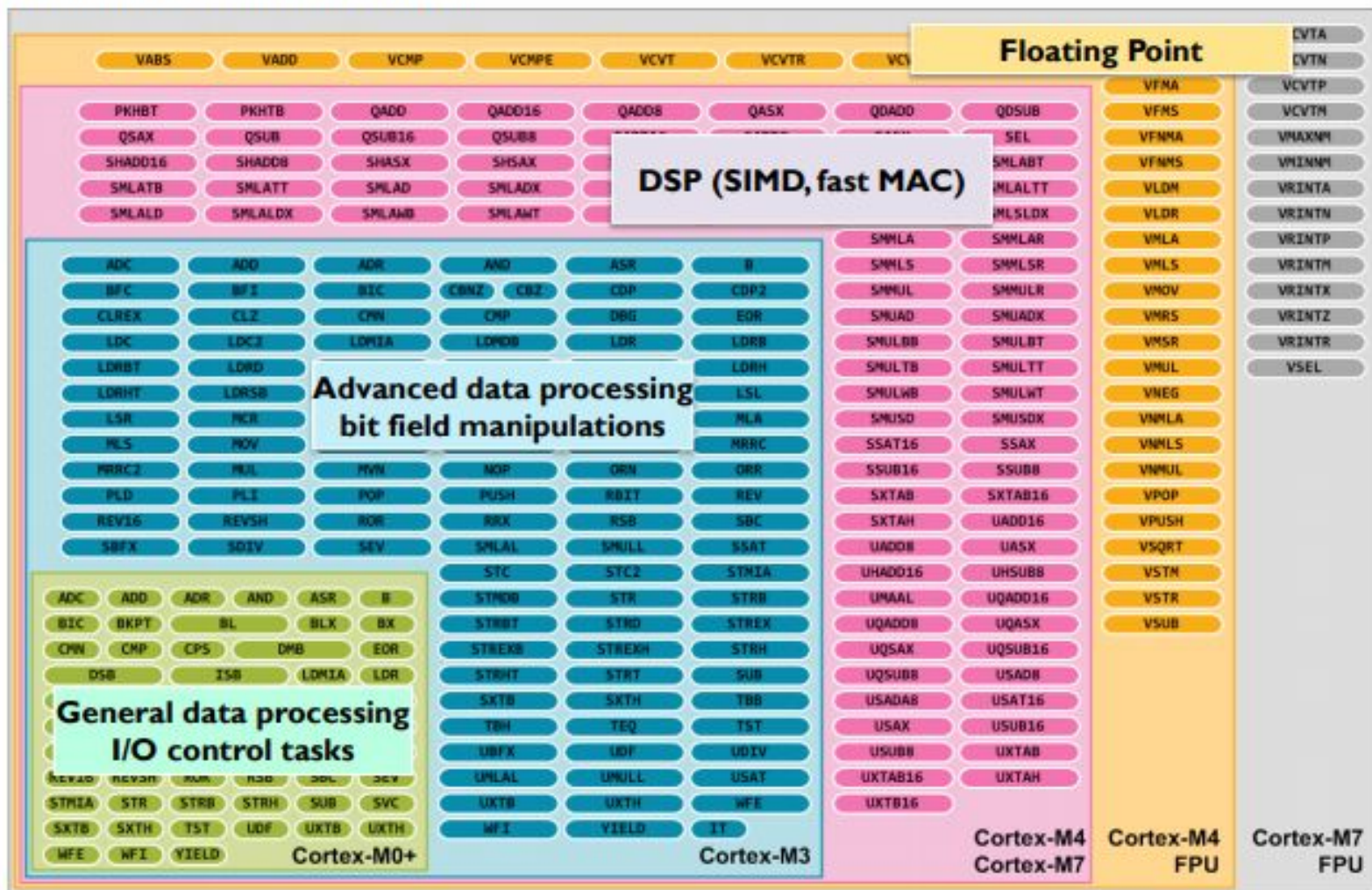
**M4 no-DSP:** Circular and bit-reversed addressing, zero overhead loops, load and store operations in parallel with math operations

**BENCHMARK:** Decodifica MP3 con menos de 10 MHz de clock (un DSP común requiere 15 MHz y uno especializado de audio 5 MHz).

**TOP**  
DSP y FPU  
SIMD

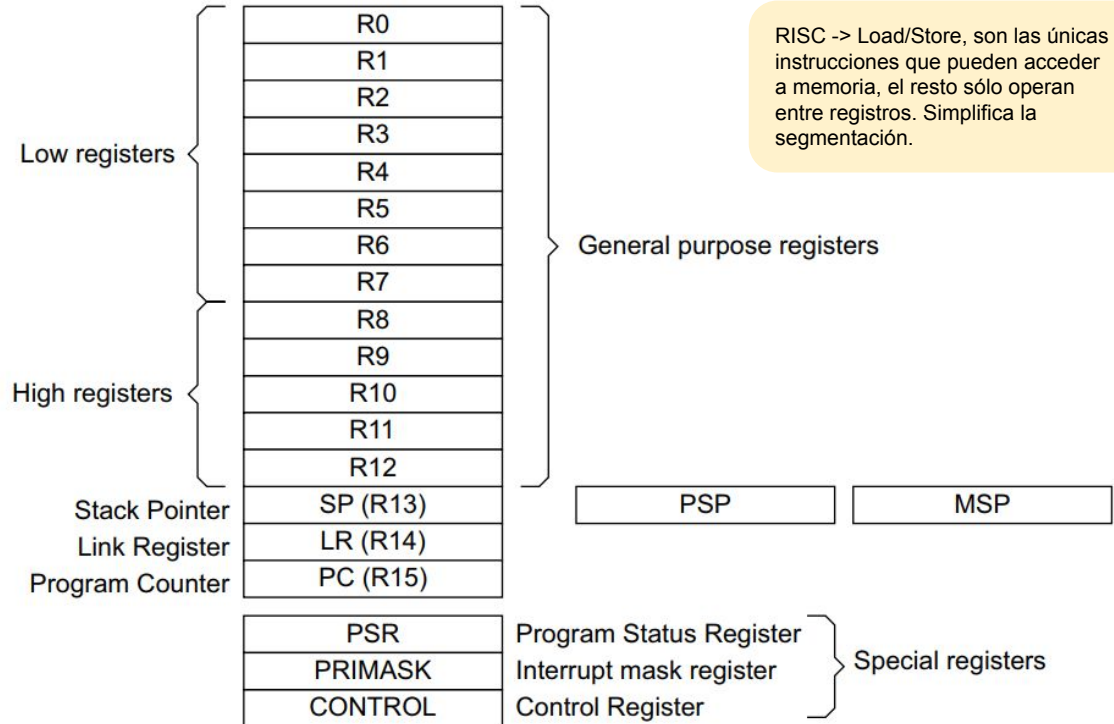
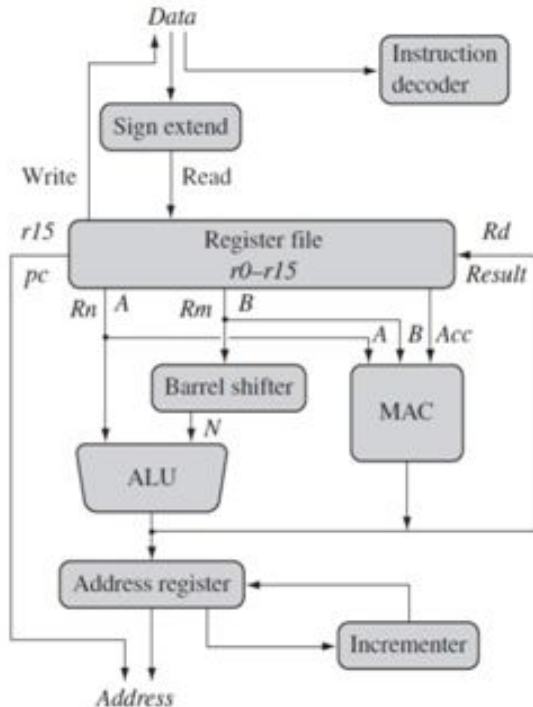
**AVANZADO**  
División, MAC  
multiciclo,  
barrel shifter,  
etc.

**MÍNIMO**  
Algunas  
instrucciones  
de 16 bits



# Arquitectura ARM básica

- RISC de 32-bit (load/store), von Neumann (4 GB memoria)
- 16 registros de 32 bits - r13/SP: stack pointer, r14/LR: link register (return address) y r15/PC + Status register: PSR
- Todos los registros pueden usarse para direccionamiento indirecto.
- 56 instrucciones base (++). Instrucciones de largo fijo de 32 bits + Thumb, algunas de 16-bits para mejorar la densidad
- **Barrel shifter en el datapath**
- **Ejecución condicional de todas las instrucciones**



RISC -> Load/Store, son las únicas instrucciones que pueden acceder a memoria, el resto sólo operan entre registros. Simplifica la segmentación.

## Repertorio de instrucciones

| Mnemonic | Instruction                                    | Action                                                                      | See Section: |
|----------|------------------------------------------------|-----------------------------------------------------------------------------|--------------|
| ADC      | Add with carry                                 | $Rd := Rn + Op2 + \text{Carry}$                                             | 4.5          |
| ADD      | Add                                            | $Rd := Rn + Op2$                                                            | 4.5          |
| AND      | AND                                            | $Rd := Rn \text{ AND } Op2$                                                 | 4.5          |
| B        | Branch                                         | $R15 := \text{address}$                                                     | 4.4          |
| BIC      | Bit Clear                                      | $Rd := Rn \text{ AND NOT } Op2$                                             | 4.5          |
| BL       | Branch with Link                               | $R14 := R15, R15 := \text{address}$                                         | 4.4          |
| BX       | Branch and Exchange                            | $R15 := Rn,$<br>$T \text{ bit} := Rn[0]$                                    | 4.3          |
| CDP      | Coprocessor Data Processing                    | (Coprocessor-specific)                                                      | 4.14         |
| CMN      | Compare Negative                               | $CPSR \text{ flags} := Rn + Op2$                                            | 4.5          |
| CMP      | Compare                                        | $CPSR \text{ flags} := Rn - Op2$                                            | 4.5          |
| EOR      | Exclusive OR                                   | $Rd := (Rn \text{ AND NOT } Op2)$<br>$\text{OR } (Op2 \text{ AND NOT } Rn)$ | 4.5          |
| LDC      | Load coprocessor from memory                   | Coprocessor load                                                            | 4.15         |
| LDM      | Load multiple registers                        | Stack manipulation (Pop)                                                    | 4.11         |
| LDR      | Load register from memory                      | $Rd := (\text{address})$                                                    | 4.9, 4.10    |
| MCR      | Move CPU register to coprocessor register      | $cRn := rRn \{<op>cRm\}$                                                    | 4.16         |
| MLA      | Multiply Accumulate                            | $Rd := (Rm * Rs) + Rn$                                                      | 4.7, 4.8     |
| MOV      | Move register or constant                      | $Rd := Op2$                                                                 | 4.5          |
| MRC      | Move from coprocessor register to CPU register | $Rn := cRn \{<op>cRm\}$                                                     | 4.16         |
| MRS      | Move PSR status/flags to register              | $Rn := PSR$                                                                 | 4.6          |
| MSR      | Move register to PSR status/flags              | $PSR := Rm$                                                                 | 4.6          |
| MUL      | Multiply                                       | $Rd := Rm * Rs$                                                             | 4.7, 4.8     |
| MVN      | Move negative register                         | $Rd := 0xFFFFFFFF \text{ EOR } Op2$                                         | 4.5          |
| ORR      | OR                                             | $Rd := Rn \text{ OR } Op2$                                                  | 4.5          |
| RSB      | Reverse Subtract                               | $Rd := Op2 - Rn$                                                            | 4.5          |
| RSC      | Reverse Subtract with Carry                    | $Rd := Op2 - Rn - 1 + \text{Carry}$                                         | 4.5          |

Table 4-1: The ARM Instruction set

MLA r1,r2,r3,r4 ; R1:=R2\*R3+R4 o sea MAC(0,4)

| Mnemonic | Instruction                          | Action                                      | See Section: |
|----------|--------------------------------------|---------------------------------------------|--------------|
| SBC      | Subtract with Carry                  | $Rd := Rn - Op2 - 1 + \text{Carry}$         | 4.5          |
| STC      | Store coprocessor register to memory | $\text{address} := cRn$                     | 4.15         |
| STM      | Store Multiple                       | Stack manipulation (Push)                   | 4.11         |
| STR      | Store register to memory             | $<\text{address}> := Rd$                    | 4.9, 4.10    |
| SUB      | Subtract                             | $Rd := Rn - Op2$                            | 4.5          |
| SWI      | Software Interrupt                   | OS call                                     | 4.13         |
| SWP      | Swap register with memory            | $Rd := [Rn], [Rn] := Rm$                    | 4.12         |
| TEQ      | Test bitwise equality                | $CPSR \text{ flags} := Rn \text{ EOR } Op2$ | 4.5          |
| TST      | Test bits                            | $CPSR \text{ flags} := Rn \text{ AND } Op2$ | 4.5          |

Table 4-1: The ARM Instruction set (Continued)

# ARM

## Codificación del repertorio de instrucciones

**<operation>{cond}{S} Rd,Rn,Op2**

<operation> A three-letter mnemonic

{cond} An optional two-letter condition code

S: set condition codes if S present

Rd: The destination register.

Rn: The first source register.

Op2: A flexible second operand.

**ADDEQ R0,R1,R2**

**ADDS R3,R5,#15**

**ANDGTS R1,R2,R3,LSR#3**

ver

[ARM Instruction Set](#)

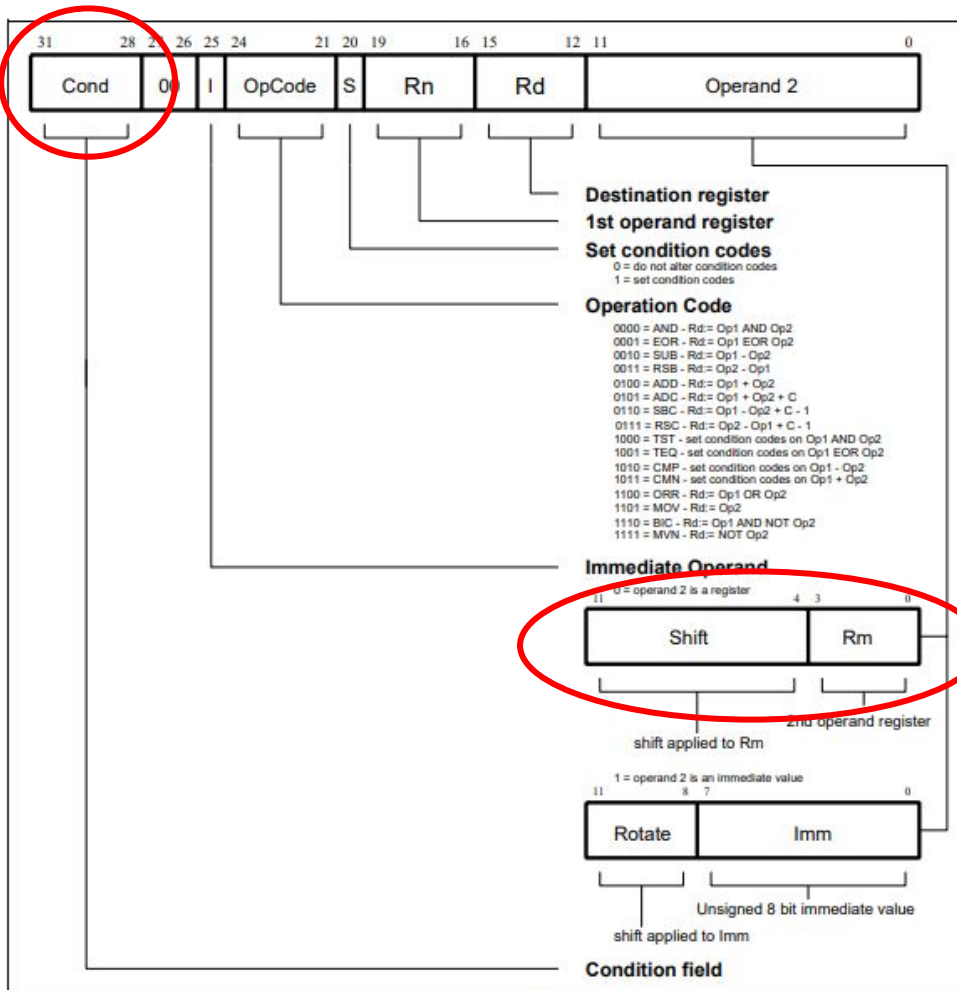


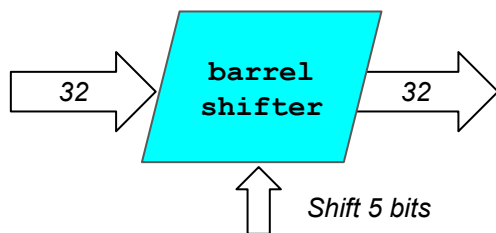
Figure 4-4: Data processing instructions

## Codificación del repertorio de instrucciones

3 3 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 9 8 7 6 5 4 3 2 1 0  
1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0

# Barrel Shifter

Circuito combinatorio puro (arreglo de multiplexores)



`MOV r3, r4, LSL #1`

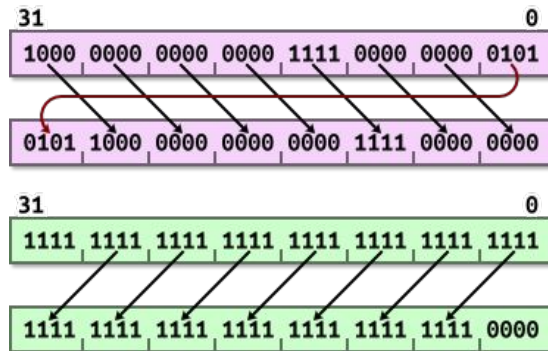
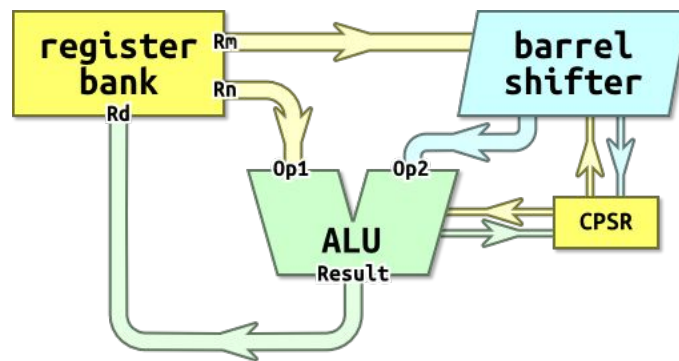
`SUB r3, r4, r5, ROR #4`

`; Multiplicar R4 por 17 (16+1)`

`ADD r4, r4, r4, LSL #4`

`; Swap the top and bottom halves of R3`

`MOV r3, r3, ROR #16`



# Ejecución condicional

Todas las instrucciones pueden ser condicionales, incluidos los saltos

```
; R1:=R2*R3+R4 condicional
```

```
MLAEQ r1,r2,r3,r4
```

```
; Multiplicar por 17 condicional
```

```
ADDGT r4,r4,r4,LSL #4
```

```
; DOS SALTOS: B y BL
```

```
back:
```

```
    ...          ;  
    B back       ; jump to label 'back'
```

```
; Llamada a función
```

```
    ...  
    BL calc      ; call 'calc', store return  
    ...          ; returns to here
```

```
calc:          ; function body  
    ADD r0, r1, r2 ; do some work here  
    MOV pc, r14   ; PC = R14 to return
```



| Code | Suffix | Flags                       | Meaning                 |
|------|--------|-----------------------------|-------------------------|
| 0000 | EQ     | Z set                       | equal                   |
| 0001 | NE     | Z clear                     | not equal               |
| 0010 | CS     | C set                       | unsigned higher or same |
| 0011 | CC     | C clear                     | unsigned lower          |
| 0100 | MI     | N set                       | negative                |
| 0101 | PL     | N clear                     | positive or zero        |
| 0110 | VS     | V set                       | overflow                |
| 0111 | VC     | V clear                     | no overflow             |
| 1000 | HI     | C set and Z clear           | unsigned higher         |
| 1001 | LS     | C clear or Z set            | unsigned lower or same  |
| 1010 | GE     | N equals V                  | greater or equal        |
| 1011 | LT     | N not equal to V            | less than               |
| 1100 | GT     | Z clear AND (N equals V)    | greater than            |
| 1101 | LE     | Z set OR (N not equal to V) | less than or equal      |
| 1110 | AL     | (ignored)                   | always                  |

Table 4-2: Condition code summary

El ejemplo típico es el **Máximo común divisor**, según el **algoritmo de Euclides**.

En C:

```
while (i != j) // Ingresa en el ciclo cuando i<j o i>j, no cuando i==j
{
    if (i > j) // Cuando i>j realiza lo siguiente
        i -= j;
    else // en otro caso, realiza lo siguiente
        j -= i;
}
```

Una estrategia más eficiente para implementar en ensamblador::

```
loop:
    // Compara i y j
    GT = i > j;
    LT = i < j;
    NE = i != j;

    // Operaciones mejoradas usando resultados de flags
    if(GT) i -= j; // Sustraer *solo* si es mayor
    if(LT) j -= i; // Sustraer *solo* si es menor
    if(NE) goto loop; // Ciclo *solo* si los valores comparados no son iguales
```

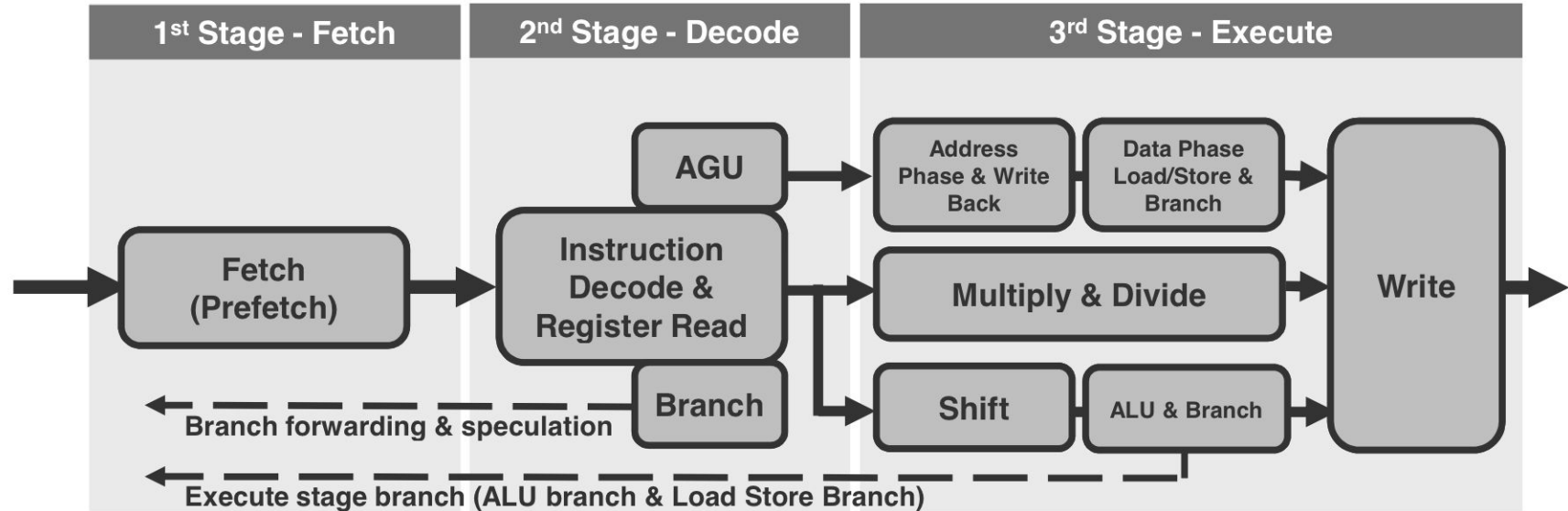
Esto en ARM se codifica en 4 instrucciones:

```
loop:  CMP    Ri, Rj          ; establece la condicion "NE" if (i != j),
                                           ;           "GT" si (i > j),
                                           ;           o  "LT" si (i < j)
      SUBGT  Ri, Ri, Rj       ; si "GT" (Mayor que), i = i-j;
      SUBLT  Rj, Rj, Ri       ; si "LT" (Menor que), j = j-i;
      BNE   loop             ; si "NE" (No igual), entonces realiza el ciclo
```

# ORGANIZACIÓN Cortex-M

3-stage pipeline core with von Neumann architecture (Harvard bus architecture (M3/M4/M7))

Speculative fetch of branch targets



M7 incluye “six-stage dual issue pipeline” (allows execution of up to two instructions in the same clock cycle), branch prediction, data caches, etc. **ARQII**

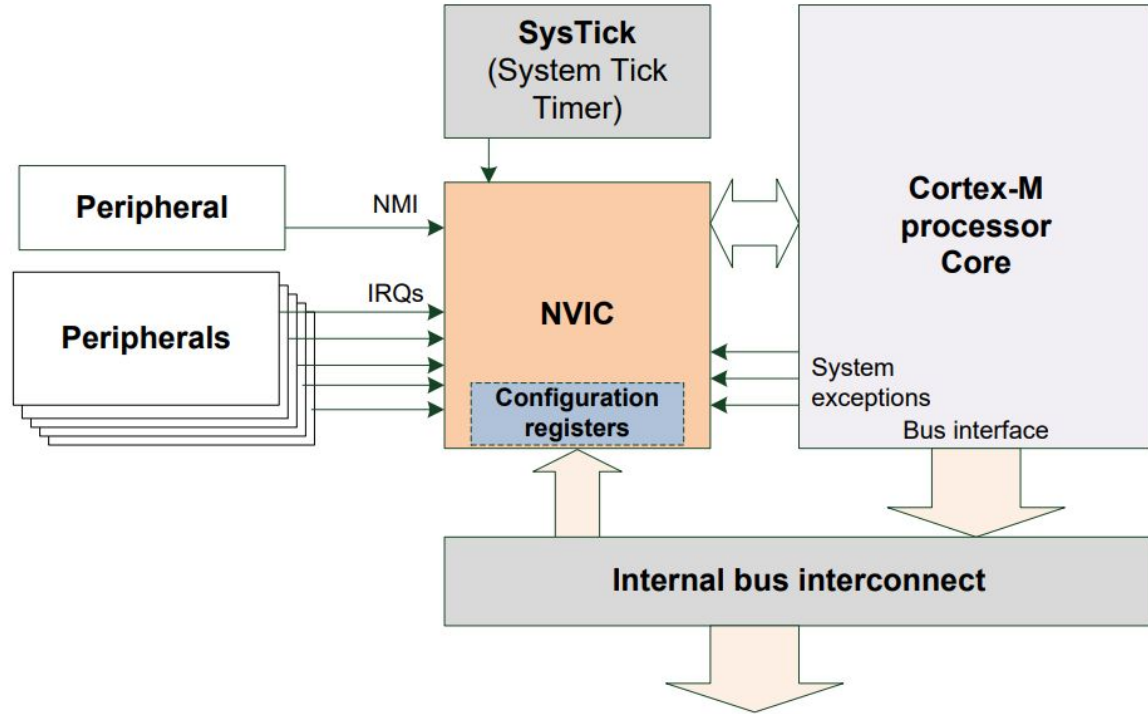
# INTERRUPT HANDLING

1 interrupción no enmascarable:  
(Non-Maskable Interrupt (INTNMI)).

16 interrupciones enmascarables y  
priorizables.

El número de interrupciones disponibles  
depende de la implementación.

*“Nested Vectored Interrupt Controller (NVIC) is tightly coupled with processor core. The NVIC supports a number of interrupt inputs from peripherals, a Non-Maskable Interrupt request, an interrupt request from a built-in timer called SysTick and a number of system exceptions. The NVIC handles the priority management and masking of these interrupt and exceptions.”*

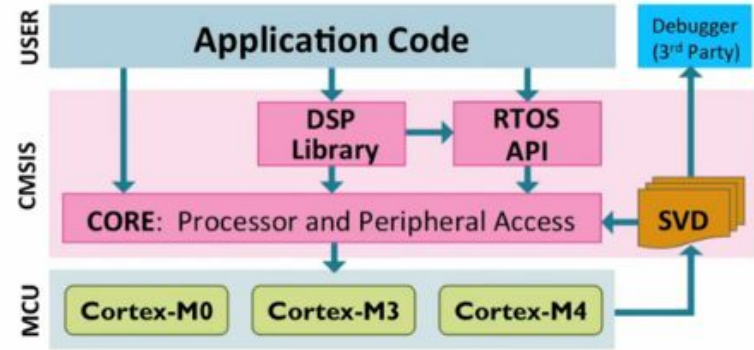


## Programación en C

Para facilitar el desarrollo de software y permitir una mejor reusabilidad de código y portabilidad, ARM desarrolló **CMSIS-Core** (Cortex Microcontroller Software Interface Standard) que provee un Hardware Abstraction Layer (HAL), estandarizado para varios recursos del procesador, como el manejo de interrupciones. Suele estar integrado en las librerías de los diversos fabricantes.

También desarrollaron una biblioteca **CMSIS-DSP** que provee varias funciones optimizadas para Cortex-M4 y que también soporta otros procesadores Cortex-M processors.

Tanto CMSIS-Core como CMSIS-DSP son gratuitas, pueden descargarse del sitio oficial de ARM ([www.arm.com/cmsis](http://www.arm.com/cmsis)), y son soportadas por la mayoría de las herramientas de desarrollo para Cortex.



## Soporte para Sistema Operativo (OS)

La arquitectura Cortex-M fue diseñada pensando en un OS. Algunas prestaciones relacionadas:

- Shadowed stack pointer.
- SVC (supervisor call) and PendSV (pendable service call) exceptions, interfaz para proveer acceso al HW a las aplicaciones.
- SysTick timer – a 24-bit down counter for generating periodic OS exception for time keeping and task management.
- Unprivileged execution level and Memory Protection Unit (MPU) in Cortex-M0+/M3/M4 and M7.

**CMSIS-RTOS** provee la interfaz a sistemas operativos de terceros:

<https://os.mbed.com/handbook/CMSIS-RTOS>

Tread, osDelay, Mutex, Semaphore, Signals, Message Queue, Memory Pool, Mail Queue, Timer, Interrupt Service Routines, Status and Error Codes, etc.

**mbed OS** es un sistema operativo desarrollado por ARM usando la interfaz CMSIS-RTOS: <https://os.mbed.com/>

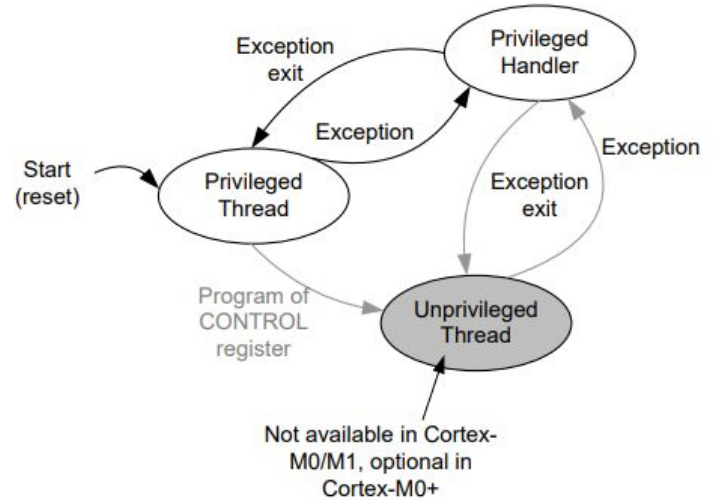
**Mbed Online Compiler** (free)

FreeRTOS <https://www.freertos.org/>

Simba <https://simba-os.readthedocs.io/en/latest/index.html>

Trampoline <https://github.com/TrampolineRTOS/trampoline>

DuinoOS <https://github.com/DuinoOS/DuinoOS>



Materia Optativa Electrónica  
**Sistemas Embebidos**

## **NOTA**

### **Sistemas operativos de tiempo real (RTOS)**

Mejor llamados “Embedded Programming Platforms”.

Beneficios de programar en el entorno de un RTOS:

- Threads scheduled by a priority based cooperative or preemptive scheduler.
- Channels for inter-thread communication (Queue, Event).
- Timers.
- Counting semaphores.
- Device drivers (SPI, UART, ...)
- A simple shell.
- Logging.
- Internet protocols (TCP, UDP, HTTP, ...).
- File systems (FAT16, SPIFFS).

# ARM Cortex-M4 DSP Instructions Compared

| CLASS          | INSTRUCTION                        | Cycle counts |           |           |
|----------------|------------------------------------|--------------|-----------|-----------|
|                |                                    | ARM9E-S      | CORTEX-M3 | Cortex-M4 |
| Arithmetic     | ALU operation (not PC)             | 1 - 2        | 1         | 1         |
|                | ALU operation to PC                | 3 - 4        | 3         | 3         |
|                | CLZ                                | 1            | 1         | 1         |
|                | QADD, QDADD, QSUB, QDSUB           | 1 - 2        | n/a       | 1         |
|                | QADD8, QADD16, QSUB8, QSUB16       | n/a          | n/a       | 1         |
|                | QDADD, QDSUB                       | n/a          | n/a       | 1         |
|                | QASX, QSAX, SASX, SSAX             | n/a          | n/a       | 1         |
|                | SHASX, SHSAX, UHASX, UHSAX         | n/a          | n/a       | 1         |
|                | SADD8, SADD16, SSUB8, SSUB16       | n/a          | n/a       | 1         |
|                | SHADD8, SHADD16, SHSUB8, SHSUB16   | n/a          | n/a       | 1         |
|                | UQADD8, UQADD16, UQSUB8, UQSUB16   | n/a          | n/a       | 1         |
|                | UHADD8, UHADD16, UHSUB8, UHSUB16   | n/a          | n/a       | 1         |
|                | UADD8, UADD16, USUB8, USUB16       | n/a          | n/a       | 1         |
|                | UQASX, UQSAX, USAX, UASX           | n/a          | n/a       | 1         |
|                | UXTAB, UXTAB16, UXTAH              | n/a          | n/a       | 1         |
|                | USAD8, USADA8                      | n/a          | n/a       | 1         |
| Multiplication | MUL, MLA                           | 2 - 3        | 1 - 2     | 1         |
|                | MULS, MLAS                         | 4            | 1 - 2     | 1         |
|                | SMULL, UMULL, SMLAL, UMLAL         | 3 - 4        | 5 - 7     | 1         |
|                | SMULBB, SMULBT, SMULTB, SMULTT     | 1 - 2        | n/a       | 1         |
|                | SMLABB, SMLBT, SMLATB, SMLATT      | 1 - 2        | n/a       | 1         |
|                | SMULWB, SMULWT, SMLAWB, SMLAWT     | 1 - 2        | n/a       | 1         |
|                | SMLALBB, SMLALBT, SMLALTB, SMLALTT | 2 - 3        | n/a       | 1         |
|                | SMLAD, SMLADX, SMLALD, SMLALDX     | n/a          | n/a       | 1         |
|                | SMLSD, SMLSDX                      | n/a          | n/a       | 1         |
|                | SMLSLD, SMLSLD                     | n/a          | n/a       | 1         |
|                | SMMLA, SMMLAR, SMMLS, SMMLSR       | n/a          | n/a       | 1         |
|                | SMMUL, SMMULR                      | n/a          | n/a       | 1         |
|                | SMUAD, SMUADX, SMUSD, SMUSDX       | n/a          | n/a       | 1         |
|                | UMAAL                              | n/a          | n/a       | 1         |
| Division       | SDIV, UDIV                         | n/a          | 2 - 12    | 2 - 12    |

Single  
cycle  
MAC

# ARM Cortex-M4 Single Cycle MAC Instructions

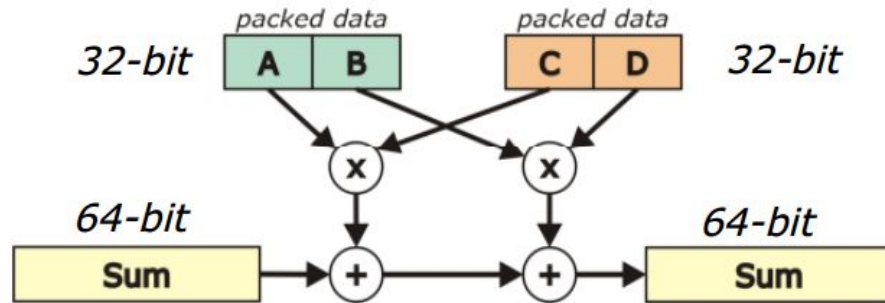
| OPERATION                                     | INSTRUCTIONS                       |
|-----------------------------------------------|------------------------------------|
| $16 \times 16 = 32$                           | SMULBB, SMULBT, SMULTB, SMULTT     |
| $16 \times 16 + 32 = 32$                      | SMLABB, SMLABT, SMLATB, SMLATT     |
| $16 \times 16 + 64 = 64$                      | SMLALBB, SMLALBT, SMLALTB, SMLALTT |
| $16 \times 32 = 32$                           | SMULWB, SMULWT                     |
| $(16 \times 32) + 32 = 32$                    | SMLAWB, SMLAWT                     |
| $(16 \times 16) \pm (16 \times 16) = 32$      | SMUAD, SMUADX, SMUSD, SMUSDX       |
| $(16 \times 16) \pm (16 \times 16) + 32 = 32$ | SMLAD, SMLADX, SMLSD, SMLSDX       |
| $(16 \times 16) \pm (16 \times 16) + 64 = 64$ | SMLALD, SMLALDX, SMLS LD, SMLS LDX |
| $32 \times 32 = 32$                           | MUL                                |
| $32 \pm (32 \times 32) = 32$                  | MLA, MLS                           |
| $32 \times 32 = 64$                           | SMULL, UMULL                       |
| $(32 \times 32) + 64 = 64$                    | SMLAL, UMLAL                       |
| $(32 \times 32) + 32 + 32 = 64$               | UMAAL                              |
| $32 \pm (32 \times 32) = 32$ (upper)          | SMMLA, SMMLAR, SMMLS, SMMLSR       |
| $(32 \times 32) = 32$ (upper)                 | SMMUL, SMMULR                      |

All the above operations are single cycle on the Cortex-M4 processor

## ARM Cortex-M4 SIMD Instructions

SIMD extensions perform multiple operations in one cycle

$$Sum = Sum + (A \times C) + (B \times D)$$



SIMD techniques operate with packed data

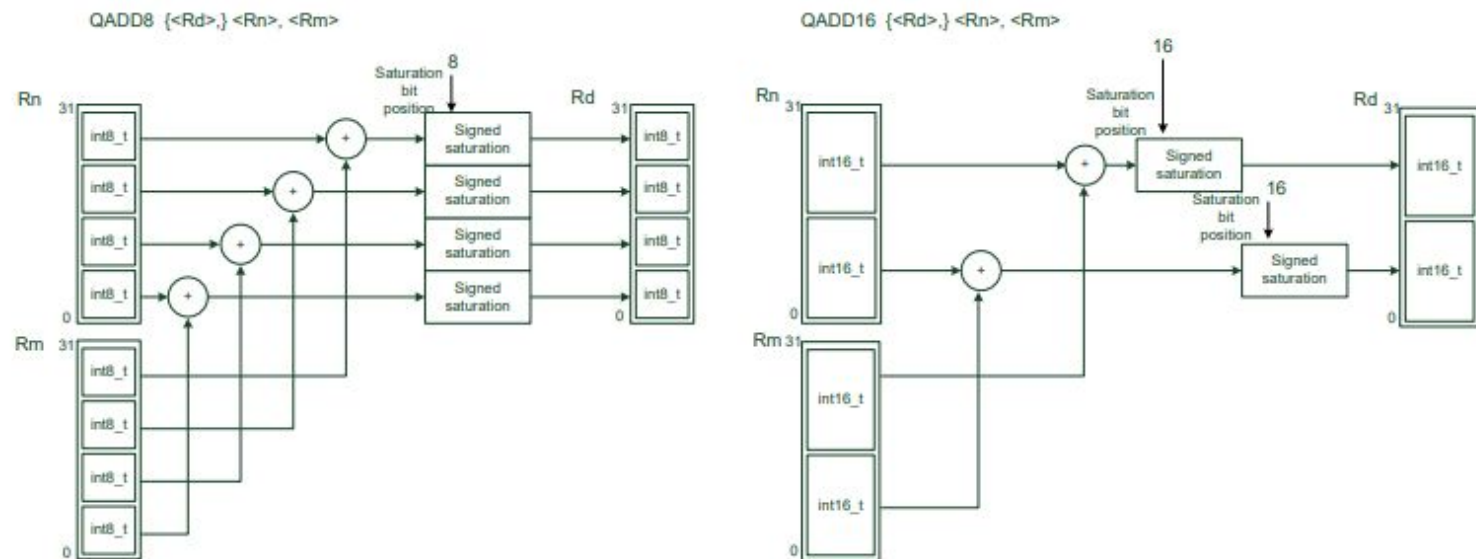


Figure 4: Example of SIMD instructions: QADD8 and QADD16

# Performance

| ARM Cortex | CoreMark/MHz | DMIPS/MHz                      |
|------------|--------------|--------------------------------|
| M7         | 5.04         | 2.14 / 2.55 / 3.23 DMIPS/MHz** |
| M4         | 3.40         | 1.25 / 1.52 / 1.91 DMIPS/MHz** |
| M3         | 3.32         | 1.25 / 1.50 / 1.89 DMIPS/MHz** |
| M0         | 2.33         | 0.87 / 1.02 / 1.27 DMIPS/MHz** |

*\*\* The first result abides by all of the “ground rules” laid out in the Dhrystone documentation, the second permits inlining of functions, not just the permitted C string libraries, while the third additionally permits simultaneous (“multi-file”) compilation. All are with the original (K&R) v2.1 of Dhrystone*

<https://www.eembc.org/coremark/scores.php>

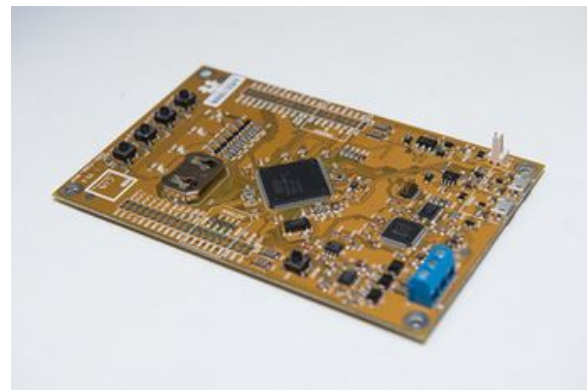
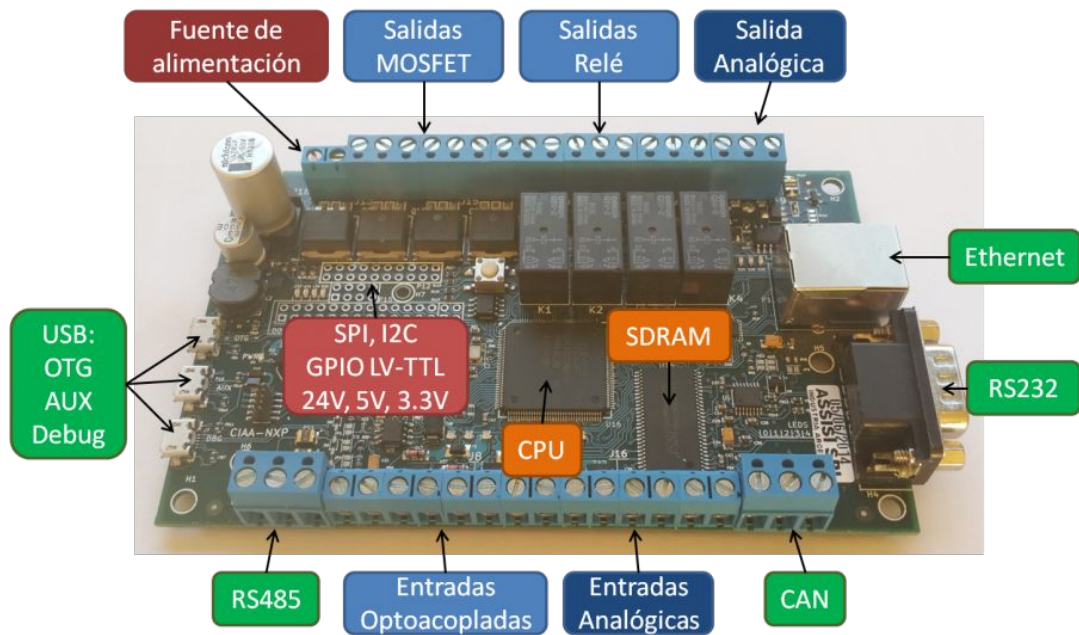
(no está STM8)

|                                                      | Compiler                   | Execution Memory                | MHz | CoreMark | CoreMark / MHz↑ | DMIPS/MHz   |
|------------------------------------------------------|----------------------------|---------------------------------|-----|----------|-----------------|-------------|
| Atmel AT89C51RE2 in X2 mode (6 clocks/machine cycle) | Keil C51 v8.18             | Internal RAM & flash (static)   | 22  | 2.36     | <b>0.11</b>     |             |
| Microchip PIC18F46K22                                | Microchip MPLAB XC8 v1.32  | Code in Flash, Data in RAM      | 64  | 7.23     | <b>0.11</b>     |             |
| STM8                                                 | Origen dudoso              |                                 | 24  | 5        | <b>0.21</b>     | <b>0.29</b> |
| Atmel ATmega644                                      | avr-gcc-4.3.2              | Flash & SRAM 20 MHz (Static)    | 20  | 10.81    | <b>0.54</b>     | <b>0.33</b> |
| Texas Instruments MSP430F5438                        | Code Composer Studio 4.1.2 | Internal flash and RAM (static) | 18  | 11.10    | <b>0.62</b>     | <b>0.31</b> |
| Microchip PIC24FJ64GA004                             | gcc 4.0.3                  | Code Flash. Data SRAM (Static)  | 32  | 23.87    | <b>0.75</b>     | <b>0.50</b> |
| STMicroelectronics STM32F103                         | GCC 4.4.1                  | Internal Flash                  | 72  | 108.26   | <b>3.32</b>     | <b>1.25</b> |

# CIAA

Computadora Industrial Abierta Argentina

<http://www.proyecto-ciaa.com.ar/>



**NXP LPC4337**  
**Cortex M4F + Cortex M0**  
**@ 204 MHz**  
1 MB Flash  
136 KB RAM

# Blue Pill

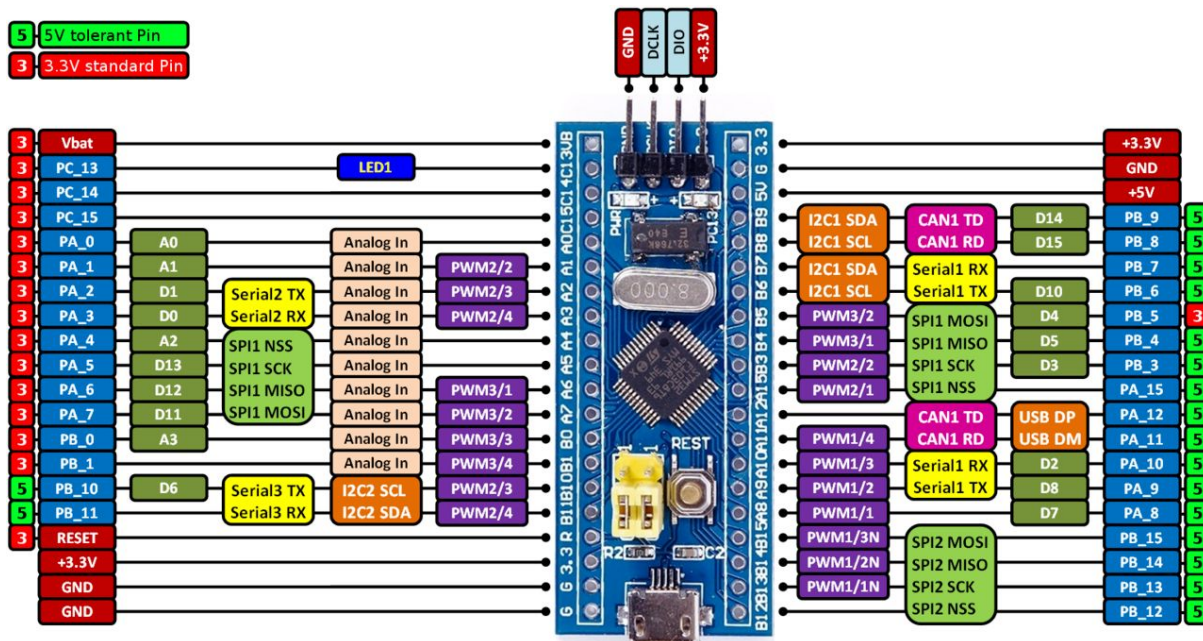
## STM32F103C8T6 - Cortex M3 @ 72 MHz

\$6500 en Mercadolibre + \$7000 Programador ST-Link V2

usd6 DigiKey x1



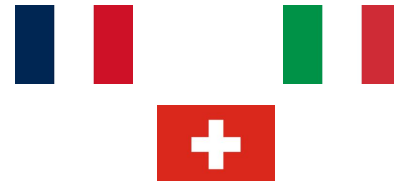
5 5V tolerant Pin  
3 3.3V standard Pin



STMicroelectronics

<https://es.wikipedia.org/wiki/STMicroelectronics>

compañía franco-italiana con sede en Suiza



# Blue Pill

## ST STM32F103C8T6 - Cortex M3 @ 72 MHz

\$6500 en Mercadolibre + \$7800 Programador ST-Link V2 (el mismo que para STM8)

| SPECS/BOARD     | STM32F103C          | ARDUINO UNO    |
|-----------------|---------------------|----------------|
| Number of Cores | 1                   | 1              |
| Architecture    | 32 Bit              | 8 Bit          |
| CPU Frequency   | 72 MHz              | 16 MHz         |
| WiFi            | NO                  | NO             |
| BLUETOOTH       | NO                  | NO             |
| RAM             | 20 KB               | 2 KB           |
| FLASH           | 64 KB               | 32 KB          |
| GPIO PINS       | 37                  | 14             |
| Busses          | SPI, I2C, UART, CAN | SPI, I2C, UART |
| ADC Pins        | 10                  | 6              |
| DAC Pins        | 0                   | 0              |

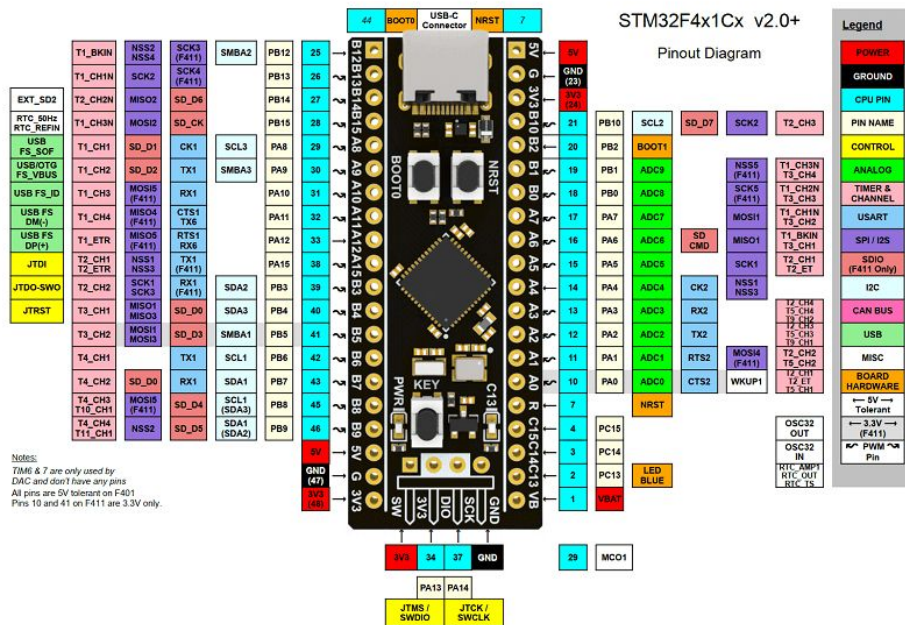
CLICK  
NOW  
SUBSCRIBE

# Black Pill

## STM32F401 - Cortex M4 @ 84 MHz

\$20,000 en Mercadolibre

usd6 DigiKey x1 (igual que un M3!)

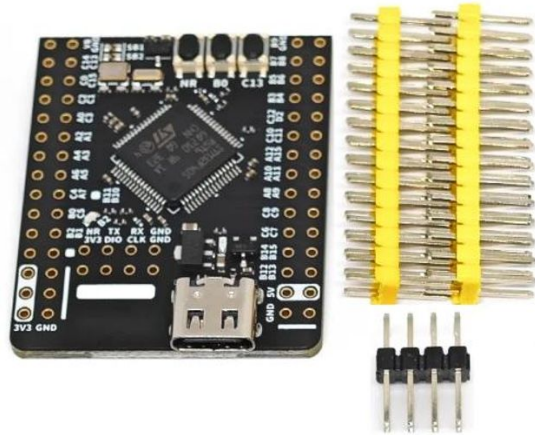


# Core Board

**STM32F405 - Cortex M4 @ 168 MHz**

Todavía están caros acá

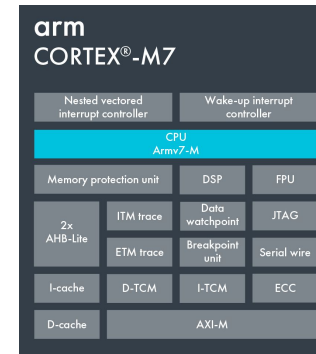
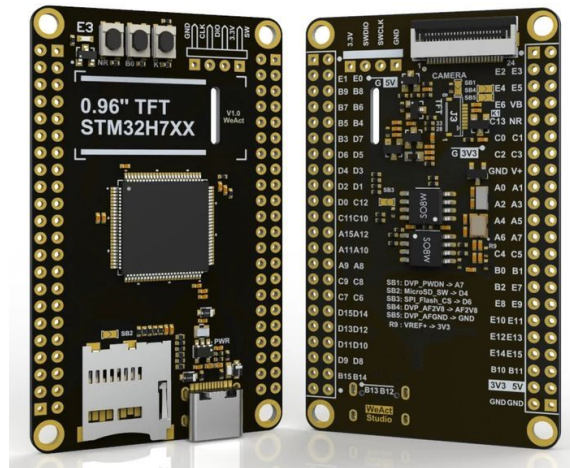
usd11 DigiKey x1



# Cortex M7

STM32H7xx 500 MHz

usd12 DigiKey x1



|                        |                                                                                                                         |
|------------------------|-------------------------------------------------------------------------------------------------------------------------|
| ISA Support            | ARMv7-M                                                                                                                 |
| DSP Extensions         | Single cycle 16/32-bit MAC<br>Single cycle dual 16-bit MAC<br>8/16-bit SIMD arithmetic<br>Hardware Divide (2-12 Cycles) |
| Floating Point Unit    | Single and double precision floating point unit<br>IEEE 754 compliant                                                   |
| Pipeline               | 6-stage 2-way superscalar + branch prediction                                                                           |
| Performance Efficiency | 5.04 CoreMark/MHz*                                                                                                      |
| Performance Efficiency | 2.14 / 2.55 / 3.23 DMIPS/MHz**                                                                                          |

**400 MHz (40 nm)**  
**Can scale up to 800MHz at 28nm**

**Mismo repertorio de instrucciones que**  
**Cortex-M4F**



The **Cortex-M33** processor is ideal for **IoT and embedded applications** that require efficient security or digital signal control.

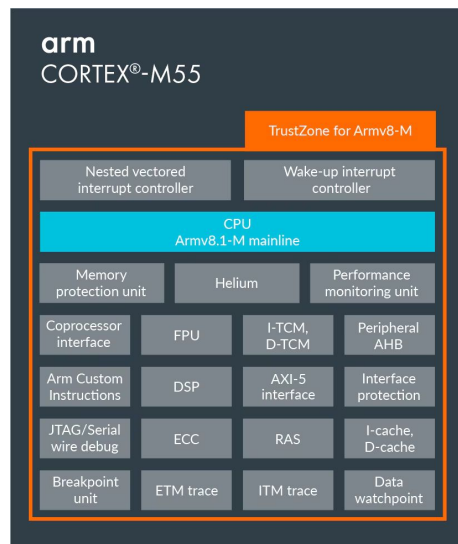
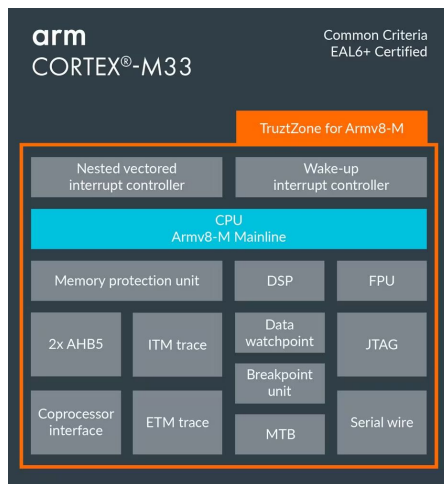
The processor has many optional features including a DSP extension, TrustZone security for hardware-enforced isolation, Arm Custom Instructions, a coprocessor interface, memory protection units, and a floating-point unit. The optional Arm Custom Instructions and coprocessor interface features open the door for customization and extensibility to further decrease the power consumption of the system in the presence of frequent compute intensive operations.

The Cortex-M33 processor with TrustZone and memory protection is certified to EAL6+ for the Common Criteria ISO 15408 standard providing security assurance for applications that require high levels of protection, such as smart cards, SIM cards, and banking cards.

The Cortex-M33 processor achieves an optimal blend between real-time determinism, energy efficiency, software productivity, and system security which opens the door for many new applications and opportunities across diverse markets.

The Arm **Cortex-M55** processor is Arm's most **AI-capable** Cortex-M processor and the first to feature Arm Helium vector processing technology.

It brings energy-efficient digital signal processing (DSP) and machine learning (ML) capabilities to the Cortex-M family. With the Cortex-M55 processor, developers can take advantage of the ease-of-use of Cortex-M, a single toolchain, optimized software libraries, and an industry-leading embedded ecosystem. The processor allows the integration of differentiating AI techniques, while staying within the system cost and energy constraints of battery powered IoT devices.



# Anexo

## ARM Cortex-M Low Power Modes

## **EN OTRA CLASE**

**Following are the architectural features which makes Cortex-M3 architecture a low power device:**

1. It has sleep mode and deep sleep mode supports, which can work with various system-design methodologies to reduce power consumption during idle period.
2. Its low gate count and design techniques reduce circuit activities in the processor to allow active power to be reduced.
3. Since Cortex-M3 has high code density, it has lowered the program size requirement. At the same time, it allows processing tasks to be completed in a short time, so that the processor can return to sleep modes as soon as possible to cut down energy use.
4. Starting from Cortex-M3 revision 2, a new feature called Wakeup Interrupt Controller (WIC) is available. This feature allows the whole processor core to be powered down, while processor states are retained and the processor can be returned to active state almost immediately when an interrupt takes place.

The following below explains the various modes available in Cortex M3 for power management.

### **Sleep Modes:**

1. During sleep mode, the system clock can be stopped, but the free-running clock input could still be running to allow the processor to be woken by an interrupt.
2. The two sleep modes are as follows:
  - a. Sleep: Indicated by the SLEEPING signal from the Cortex-M3 processor
  - b. Deep sleep: Indicated by the SLEEPDEEP signal from the Cortex-M3 processor
3. The sleep modes are invoked by Wait-For-Interrupt (WFI) or Wait-For-Event (WFE) instructions.
4. The events for invoking the sleep modes can be interrupts, a previously triggered interrupt, or an external event signal pulse via the Receive Event (RXEV) signal.
5. To decide which sleep mode is to be invoked in case of an event can be set by setting the SLEEPDEEP bit field of the Nested Vectored Interrupt Controller (NVIC) control register
6. The sleep mode operation of the processor depends on chip design. In some cases the clock signals can be stopped to reduce power consumption. The chip can also be designed to shut down part of the chip to further reduce power, or it is also possible that a design can shut down the chip completely. In a case where the chip is shut down completely, the only way to wake the system from sleep is via a system reset.

### **Sleep-On-Exit Feature:**

1. Processor can be programmed to go back to sleep automatically after the interrupt routine exit. In this way, we can make the core sleep all the time unless an interrupt needs to be served
2. To use this feature, we need to set the SLEEPONEXIT bit in the System Control register.
3. If the Sleep-On-Exit feature is enabled, the processor can enter sleep at any exception return to thread level, even if no WFE/WFI instruction is executed.

### **Wake-up Interrupt Controller:**

1. A new unit called the Wakeup Interrupt Controller (WIC) is available as an optional component. This controller is coupled to the existing NVIC and is used to generate a wakeup request when an interrupt arrives.
2. By using the technology called State Retention Power Gating (SRPG) and WIC together, most portions of the Cortex-M3 processor can be powered down during deep sleep, leaving a small amount of logic for state retention

# Anexo: ARM Cortex-M Low Power Modes

Cada microcontrolador basado en el procesador Arm Cortex-M tendrá al menos tres **Power Modes: Run, Sleep y Deep Sleep**. El modo Run es cuando el procesador está completamente encendido. El modo Sleep detendrá el reloj de la CPU pero dejará operativos el reloj del sistema, la memoria flash y los periféricos. El modo Deep Sleep no sólo detendrá el reloj de la CPU, sino que también apagará el reloj del sistema, la memoria flash y el PLL.

El fabricante de microcontroladores tiene la capacidad de personalizar completamente sus módulos de bajo consumo. Por ejemplo, los procesadores NXP Kinetis-L no solo tienen los modos estándar de bajo consumo Cortex-M, sino también modos como Low-Leakage Sleep Mode y Very Low Leakage Sleep Mode, que consumen sólo micro o nanoamperes de corriente.

Es importante tener en cuenta que cuanto más profundamente duerme el microcontrolador, más cerca está de apagarse por completo. Es importante darse cuenta de esto porque a medida que utiliza modos de suspensión cada vez más profundos, el tiempo necesario para volver a encender el microcontrolador y comenzar a ejecutar instrucciones puede aumentar dramáticamente. Algunos procesadores que ofrecen modos de energía muy profundos requieren la misma cantidad de tiempo que la secuencia de inicio del procesador para volver a estar operativos. Dependiendo de su aplicación, esto podría tener efectos significativos en el rendimiento en tiempo real del sistema.

## Instrucción WFI (Wait for Interrupt)

When the WFI instruction is encountered, the processor will immediately move into the low power sleep mode that is configured in the **System Control Block (SCB)** unless there is a pending interrupt. Once the processor is asleep, it will wake-up when an interrupt is triggered. Some processors may even have an optional Wakeup-Interrupt-Controller which allows a developer to configure what sources are allowed to wake up the processor. This provides a developer with finer grain control over how long their system can remain in the lowest power sleep mode.

Within the **SCR register** there is a **SLEEPDEEP** bit that if set to 1 will put the processor into a deep sleep mode. If the bit is cleared to 0, the processor will go into the sleep mode. As I mentioned earlier, the sleep modes are often enhanced by the silicon vendor so there will also be additional registers that need to be configured based on who designed your microcontroller.

# System Control Register

The SCR controls features of entry to and exit from low power state. See the register summary in [Table 4.12](#) for its attributes. The bit assignments are:

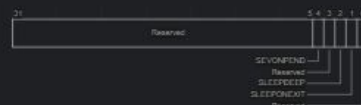


Table 4.19. SCR bit assignments

| Bits   | Name        | Function                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [31:5] | -           | Reserved.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| [4]    | SEVONPEND   | <p>Send Event on Pending bit:</p> <p>0 = only enabled interrupts or events can wakeup the processor, disabled interrupts are excluded<br/>1 = enabled events and all interrupts, including disabled interrupts, can wakeup the processor.</p> <p>When an event or interrupt enters pending state, the event signal wakes up the processor from WFE. If the processor is not waiting for an event, the event is registered and affects the next WFE.</p> <p>The processor also wakes up on execution of an <a href="#">SEV</a> instruction or an external event.</p> |
| [3]    | -           | Reserved.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| [2]    | SLEEPDEEP   | <p>Controls whether the processor uses sleep or deep sleep as its low power mode:</p> <p>0 = sleep<br/>1 = deep sleep.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| [1]    | SLEEPONEXIT | <p>Indicates sleep-on-exit when returning from Handler mode to Thread mode:</p> <p>0 = do not sleep when returning to Thread mode<br/>1 = enter sleep, or deep sleep, on return from an ISR.</p> <p>Setting this bit to 1 enables an interrupt driven application to avoid returning to an empty main application.</p>                                                                                                                                                                                                                                              |
| [0]    | -           | Reserved.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

# Low Power Modes in STM32

<https://controllerstech.com/low-power-modes-in-stm32/>

*Documentado perfectamente  
en el Reference Manual del  
STM32*

The device requires a 2.0-to-3.6 V operating voltage supply ( $V_{DD}$ ). An embedded regulator is used to supply the internal 1.8 V digital power.

The real-time clock (RTC) and backup registers can be powered from the VBAT voltage when the main  $V_{DD}$  supply is powered off.

By default, the microcontroller is in Run mode after a system or a power Reset. Several low power modes are available to save power when the CPU does not need to be kept running, for example when waiting for an external event. It is up to the user to select the mode that gives the best compromise between low-power consumption, short startup time and available wakeup sources.

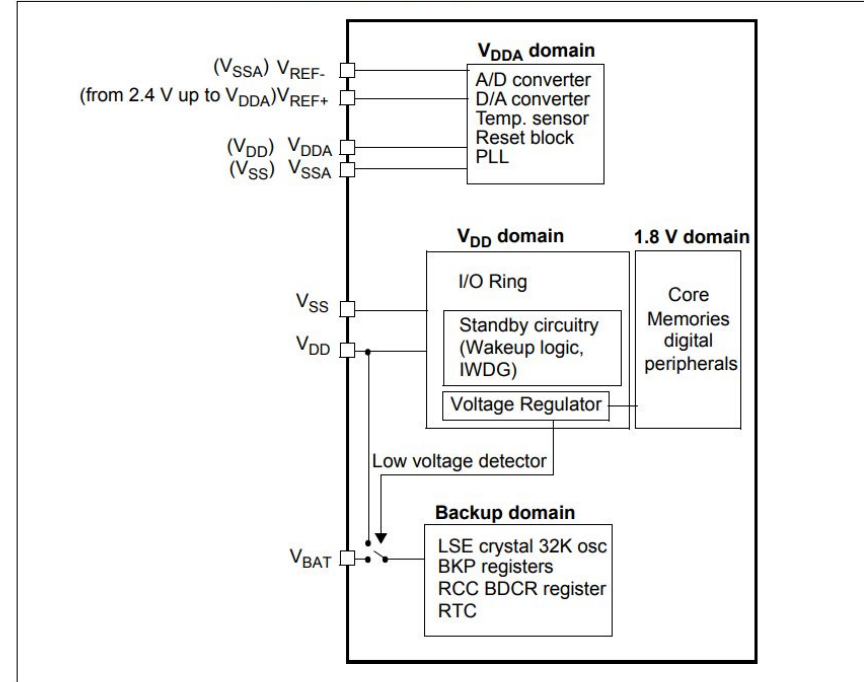
The STM32F10xxx devices feature three low-power modes:

- **Sleep mode** (CPU clock off, all peripherals including Cortex®-M3 core peripherals like NVIC, SysTick, etc. are kept running)
- **Stop mode** (all clocks are stopped)
- **Standby mode** (1.8V domain powered-off)

In addition, the power consumption in Run mode can be reduced by one of the following means:

- Slowing down the system clocks
- Gating the clocks to the APB and AHB peripherals when they are unused.

Figure 4. Power supply overview



1.  $V_{DDA}$  and  $V_{SSA}$  must be connected to  $V_{DD}$  and  $V_{SS}$ , respectively.

## SLEEP MODE

In this mode, CPU CLK is turned OFF and there is no effect on other clocks or analog clock sources. The current consumption is HIGHEST in this mode, compared to other Low Power Modes. In order to enter the SLEEP MODE, we must disable the systick interrupt first, or else this interrupt will wake the MCU every time the interrupt gets triggered.

```
HAL_SuspendTick();
```

Next, we will enter the sleep mode by executing the WFI (Wait For Interrupt), or WFE (Wait For Event) instructions. If the WFI instruction was used to enter the SLEEP MODE, any peripheral interrupt acknowledged by the NVIC can wake up the device.

```
HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
```

The device will wakeup whenever any interrupt is triggered.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {  
    HAL_ResumeTick();  
}
```

Inside the callback function we can resume the systick, so that we can use the delay function again for the rest of the code.

**SLEEPONEXIT:** This is another feature available in SLEEP MODE, where the MCU will wake up when the interrupt is triggered, it will process the ISR, and go back to sleep when the ISR exits. This is useful when we want the controller to run only in the interrupt mode.

Ver ejemplo y video con mediciones: Run 6.7 mA -> Sleep 1.8 mA

## STOP MODE

In Stop mode, all clocks in the 1.2 V domain are stopped, the PLLs, the HSI and the HSE RC oscillators are disabled. Internal SRAM and register contents are preserved. STOP MODE have may different categories depending on what should be turned off. See STM32F446RE reference manual. Just like sleep mode, in order to enter the STOP MODE, we must disable the systick interrupt, or else this interrupt will wake the MCU every time the interrupt gets triggered.

```
HAL_SuspendTick();
```

Next, we will enter the sleep mode by executing the WFI (Wait For Interrupt), or WFE (Wait For Event) instructions. If the WFI instruction was used to enter the STOP MODE, EXTI, Independent watchdog (IWDG), or RTC can wake up the device. Also I am using the first mode as shown in the picture above i.e. The main Regulator will be turned off and only the LOW Power Regulator will be running.ce.

```
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
```

We have entered the STOP MODE using the WFI instruction, so the device will wakeup whenever any interrupt is triggered by an EXTI, Independent watchdog (IWDG), or RTC. We must reconfigure the SYSTEM CLOCKS after wakeup, as they were disabled when entering the STOP MODE. Also don't forget to resume the systick.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {  
    if(GPIO_Pin == GPIO_PIN_13) {  
        SystemClock_Config ();  
        HAL_ResumeTick();  
    }  
}
```

**SLEEPONEXIT:** idem

Ver ejemplo y video con mediciones: Stop xx mA

| Table 17. Stop operating modes |                          |           |           |          |          |                                                                                                                                              |
|--------------------------------|--------------------------|-----------|-----------|----------|----------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Voltage Regulator Mode         | UDEN[1:0] bits           | MRUDS bit | LPUDS bit | LPDS bit | FPDS bit | Wakeup latency                                                                                                                               |
| Normal mode                    | STOP MR (Main Regulator) | -         | 0         | -        | 0        | HSI RC startup time                                                                                                                          |
|                                | STOP MR-FPD              | -         | 0         | -        | 0        | Flash wakeup time from power-down mode                                                                                                       |
|                                | STOP LP                  | -         | 0         | 0        | 1        | HSI RC startup time + regulator wakeup time from LP mode                                                                                     |
|                                | STOP LP-FPD              | -         | -         | 0        | 1        | Flash wakeup time from power-down mode + regulator wakeup time from LP mode                                                                  |
| Under-drive Mode               | STOP UMR-FPD             | 3         | 1         | -        | 0        | HSI RC startup time + Flash wakeup time from power-down mode + Main regulator wakeup time from under-drive mode + Core logic to nominal mode |
|                                | STOP ULP-FPD             | 3         | -         | 1        | 1        | HSI RC startup time + Flash wakeup time from power-down mode + regulator wakeup time from LP under-drive mode + Core logic to nominal mode   |

## STANDBY MODE

The Standby mode allows to achieve the lowest power consumption. It is based on the Cortex®-M4 with FPU deepsleep mode, with the voltage regulator disabled. The 1.2 V domain is consequently powered off. The PLLs, the HSI oscillator and the HSE oscillator are also switched off.

Before entering the STANDBY MODE, we must disable the wakeup flags as shown below

```
/* Clear the WU FLAG */
HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);
/* clear the RTC Wake UP (WU) flag */
HAL_RTC_WAKEUPTIMER_CLEAR_FLAG(&hrtc, RTC_FLAG_WUTF);
```

Next, enable the wakeup pin, or the RTC periodic wakeup (if you want to use RTC)

```
HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1);
```

and finally enter the STANDBY MODE

```
HAL_PWR_EnterSTANDBYMode();
```

The standby wakeup is same as a system RESET. The entire code runs from the beginning just as if it was a RESET. The only difference between a reset and a STANDBY wakeup is that, when the MCU wakes up, The SBF status flag in the PWR power control/status register (PWR\_CSR) is set. The wakeup can be triggered by WKUP pin rising edge, RTC alarm (Alarm A and Alarm B), RTC wakeup, tamper event, time stamp event, external reset in NRST pin, IWDG reset.

See entire code for the STANDBY MODE

Ver ejemplo y video con mediciones: Standby 2 uA

# Low Power Modes in STM32

<https://controllerstech.com/low-power-modes-in-stm32/>

*“The STM32U3 series offers greater power efficiency in run mode compared to previous product generations. It is the first STM32 based on near-threshold design which drastically reduces the dynamic consumption of the final application. This product represents the market-leader in term of efficiency with 117 Coremark/mW and a static mode < 2  $\mu$ A.*

<https://www.st.com/en/microcontrollers-microprocessors/stm32u3-series.html>

# Anexo

## ARQUITECTURAS DSP

# DSP: Digital Signal Processor

Un DSP no es más que un **microprocesador especializado**. Tanto en su repertorio de instrucciones como en su organización. La tecnología utilizada es la misma que para los procesadores de propósitos generales.

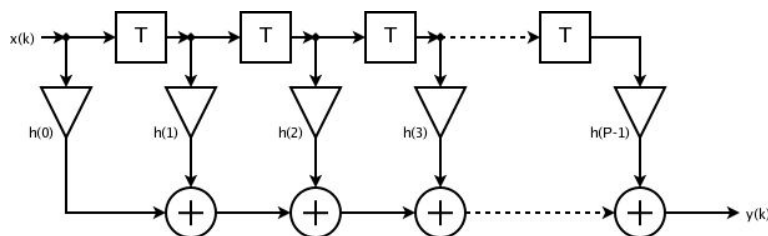
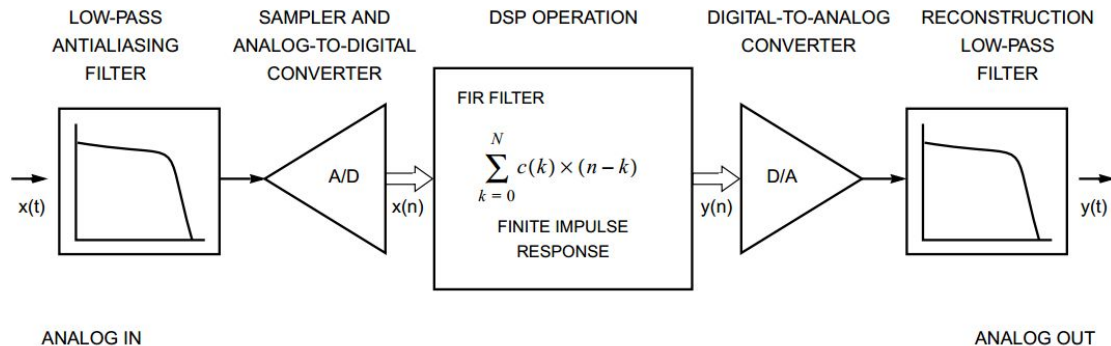
Se trata de una arquitectura optimizada para las necesidades del procesamiento digital de señales. Optimizada tanto en capacidad de cómputo como en eficiencia energética.

Aplicaciones en audio, comunicaciones, radar, reconocimiento de voz, procesamiento de imágenes médicas, televisión digital, industria automotriz, etc.

***Phil Lapsley et al***, DSP Processor Fundamentals, *IEEE Press* 1997

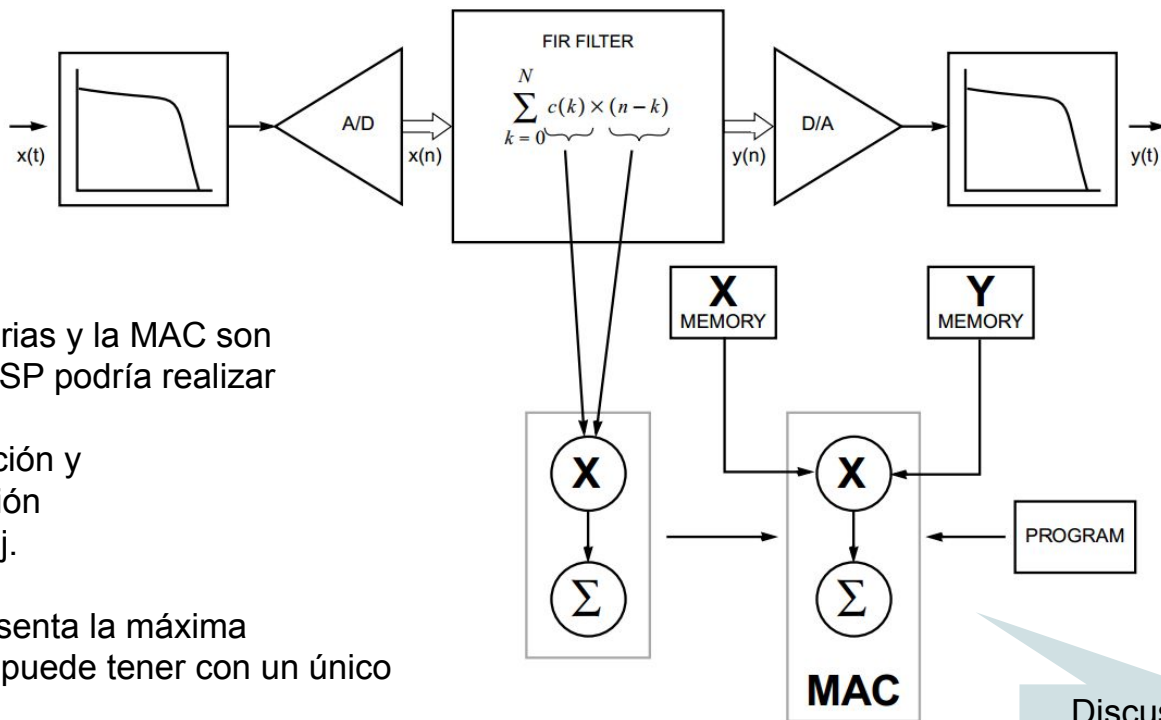
***Motorola***, DSP56000 User Manual, 1994

# Procesamiento digital de señales



Filtro FIR (respuesta finita al impulso),  $N$  es el orden del filtro

# Arquitectura Harvard + MAC



Como las dos memorias y la MAC son independientes, el DSP podría realizar

- dos cargas,
- una multiplicación y
- una acumulación

en cada ciclo de reloj.

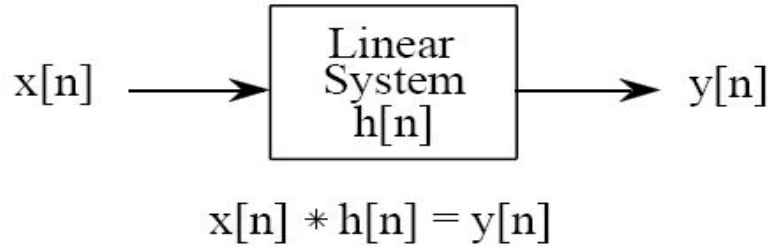
Este esquema representa la máxima performance que se puede tener con un único multiplicador.

**ESTRATEGIA: FAVORECER EL CASO MÁS FRECUENTE**

Discusión sobre los beneficios y diseño del acumulador

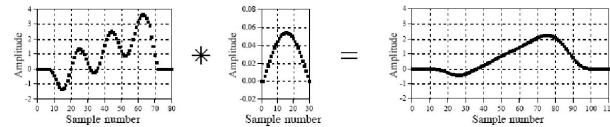
# Suma de productos

## Convolución de secuencias discretas

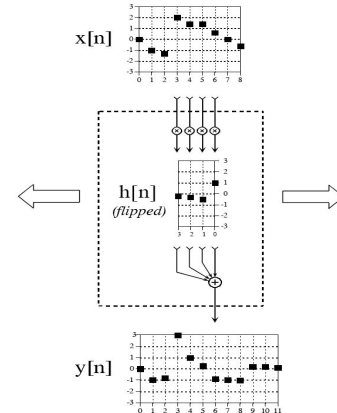
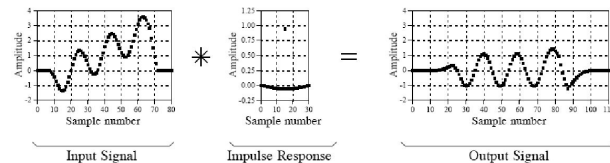


$$y[i] = \sum_{j=0}^{M-1} h[j] x[i-j]$$

a. Low-pass Filter



b. High-pass Filter



# Consideraciones generales de diseño de la solución

- Tipos de representación numérica:  
punto fijo versus punto flotante: rango y precisión
- Algoritmo a implementar
- Frecuencia de muestreo  $fm$  necesaria
- Frecuencia de reloj  $fc$  del DSP
- Relación  $fc/fm$  (“cantidad de hardware disponible”)

El problema

El dispositivo



Cuántas instrucciones del DSP puedo ejecutar entre dos muestras de la señal

# DETALLES DE DISEÑO

1. **Representación numérica** -> Punto fijo  $[-1, +1]$  vs. punto flotante.
2. **Aritmética** -> Datapath incluye acumulador, shifter normalizador, saturación y redondeo.
3. **Sistema de memoria** -> Harvard, multiple access.
4. **Acceso a los datos** -> AGU doble (address generation unit) y modos de direccionamiento especiales (circular, bit reversed).
5. **Control** -> Set de instrucciones especializado (MAC, loop, etc.).

Incluyen **segmentación** profunda, características **superescalares** y **VLIW** (paralelismo)

Compiladores (C)